

Pasture Growth Analysis Using Machine Learning

Varun Vikas, Jaiswal (N11736089)
IFN703 Assessment 3

Executive Summary

This project focuses on analyzing pasture growth using advanced machine learning techniques, with the main goal of predicting Total Seasonal Dry Matter (TSDM)—a critical measure of pasture productivity. The project explores how temporal and environmental factors like rainfall, temperature, humidity, and solar radiation influence pasture growth, aiming to optimize livestock feeding strategies and promote sustainable farm management. These insights provide actionable information for grazing management, supplementary feeding, and efficient resource allocation.

Random Forest and XGBoost were chosen as the primary models for their robustness in handling complex datasets and ability to capture non-linear relationships between variables. Key steps in the project included thorough data preprocessing, such as standardizing scales, identifying and handling outliers, and engineering features to better represent relationships within the data. Using a month-based approach that incorporates historical averages, the models provided insights into future TSDM trends.

Results demonstrated that both models effectively predict pasture growth: Random Forest achieved an R^2 of 0.717, explaining 72% of the variance in TSDM_MEAN, while XGBoost achieved an R^2 of 0.664. Additionally, the month-based historical averages approach helped generate valuable projections for future TSDM, proving the models utility in forecasting for dynamic agricultural management. Overall, the findings support the role of machine learning in livestock management by enabling data-driven decisions that contribute to healthier livestock, better meat quality, and sustainable farming.

Introduction

Pasture growth plays a crucial role in livestock management, serving as the primary source of nutrition for grazing animals and directly impacting their health, productivity, and meat quality. For farmers, accurately forecasting pasture productivity is essential for informed decision-making in areas like grazing rotation, feeding schedules, and efficient resource allocation. Total Seasonal Dry Matter (TSDM) is widely used as a key measure of pasture productivity, representing the available biomass that sustains livestock. Predicting TSDM helps optimize farm management, supporting both animal welfare and profitability.

Machine learning (ML) techniques, particularly Random Forest and XGBoost models, have proven effective in enhancing predictive capabilities in agricultural applications. These models excel at identifying complex, non-linear relationships within datasets, especially when analyzing environmental and temporal variables such as rainfall, temperature, humidity, and solar radiation.

These variables are vital in determining pasture growth rates, as they closely relate to seasonal patterns and environmental fluctuations. By employing these advanced ML models, this study aims to improve the accuracy of TSDM predictions, thus enabling data-driven decisions for sustainable livestock management.

Background of the Study

Pasture management is foundational to livestock health, as it affects growth rates, reproductive performance, and overall well-being. Variations in pasture productivity can lead to feed shortages, affecting livestock health and, ultimately, farm profitability. The ability to accurately predict pasture growth enables farmers to adjust grazing rotations, provide supplemental feed when necessary, and optimize stocking rates. Sustainable pasture management not only supports animal health but also helps preserve soil quality, reduce erosion, and maintain ecological balance.

As a widely accepted indicator of pasture productivity, TSDM measures the total dry matter produced over a growing season, which serves as a proxy for the amount of available biomass for livestock. Monitoring TSDM allows farmers to gain valuable insights into growth patterns, making it possible to adapt their practices according to forecasted feed availability. By improving the accuracy of TSDM predictions, this study seeks to empower farmers with data-driven tools to optimize livestock nutrition and farm efficiency.

Main Objective

The primary objective of this study is to analyze how environmental and temporal factors—including rainfall, temperature, humidity, and solar radiation—affect TSDM. By developing robust ML models, specifically Random Forest and XGBoost, the study aims to provide a reliable forecasting tool for pasture growth. Accurate predictions will enable farmers to manage grazing rotations effectively, allocate feed resources efficiently, and respond proactively to environmental changes. This research ultimately seeks to support sustainable and data-driven livestock management, helping to reduce uncertainties and strengthen the resilience of agricultural practices.

Context for Results Discussion

This study leverages two ML models, Random Forest and XGBoost, known for their capability to handle large, complex datasets and capture intricate non-linear relationships. Random Forest combines multiple decision trees to enhance predictive accuracy, making it particularly effective when analyzing datasets with numerous interacting features. XGBoost, an advanced gradient-boosting model, sequentially builds and optimizes decision trees, allowing it to achieve high precision by correcting errors iteratively.

These models are well-suited for analyzing the environmental and temporal factors influencing TSDM, including key variables such as rainfall, temperature, humidity, and solar radiation. By accurately forecasting TSDM, Random Forest and XGBoost facilitate practical, data-driven insights for farm management, enabling farmers to make well-informed decisions regarding grazing schedules, livestock feeding, and resource allocation. This study bridges complex environmental data with actionable strategies, fostering sustainability and productivity in agriculture.

Definition of Specialist Terms

Total Seasonal Dry Matter (TSDM): The total weight of dry matter produced over a growing season, indicating available biomass for livestock feeding.

Literature Review

Background and Context

In recent years, machine learning (ML) techniques have gained prominence in agriculture, particularly in optimizing crop and pasture yield predictions. The growing availability of environmental and temporal datasets has facilitated the integration of various factors, such as climatic and management data, into predictive models to enhance sustainable farming practices. Numerous studies emphasize the effectiveness of ML models, including Random Forest (RF) and XGBoost, in capturing complex interactions among variables to produce accurate agricultural predictions. This review highlights recent findings on the application of ML in agriculture, focusing on their effectiveness, limitations, and implications for improving predictive capabilities.

Key Findings from the Literature

Several studies have explored the utility and challenges of ML models in agriculture. For instance, Silva et al. (2023) assessed the explanatory and predictive power of RF models on large-scale agricultural datasets, noting that while RF effectively identifies relationships within complex datasets, its predictive capability diminishes when applied across varying geographic and temporal conditions. This finding suggests a need for cautious application, especially in agronomic contexts where data heterogeneity is present.

Another prominent ML model, XGBoost, has demonstrated robust predictive capabilities in agricultural settings. In a study by Ravi and Baranidharan (2020), XGBoost outperformed traditional models, such as Support Vector Regression and Decision Trees, in crop yield prediction, achieving an R^2 of 0.9391. The model's efficiency in handling interactions across multiple variables and large datasets underscores its suitability for structured agricultural applications.

Studies specific to pasture growth, such as De Rosa et al. (2021), emphasize the importance of incorporating environmental variables, including post-grazing biomass and climate data, into ML models. Their research integrated statistical methods with ML algorithms to improve prediction accuracy, illustrating the relevance of both temporal and environmental factors in dynamic systems like pastures. Such approaches underscore the importance of a multi-faceted dataset when modeling agricultural outcomes.

An innovative approach integrating Genetic Algorithms, ARIMA, and Random Forest was explored by Guo (2024), which optimized feature selection and reduced overfitting, enhancing yield forecasting accuracy. This hybrid model leveraged reinforcement learning to adapt to complex agricultural datasets, demonstrating the benefits of combining feature selection with ML models for improved prediction stability.

Additional research by Wang et al. (2022) emphasized the critical role of handling temporal variability using multi-temporal ML models. Their findings support the utility of combining multi-temporal data sources with ML techniques, particularly RF and XGBoost, to capture detailed growth patterns, thereby enhancing the robustness of agricultural predictions in fluctuating environments.

Methods and Techniques Reviewed

The literature review indicates the critical role of combining machine learning techniques, statistical models, and advanced optimization algorithms in agricultural research. Key methodologies reviewed include:

- **Random Forest:** Effective in identifying complex patterns within large datasets but limited by data quality and external variability .
- **XGBoost:** Known for its scalability and high predictive accuracy, particularly in structured datasets with multiple interacting variables.
- **Genetic Algorithm-based ARIMA and Reinforced Random Forest:** Incorporating reinforcement learning for better attribute selection and reduced overfitting, leading to improved forecasting capabilities.

Analysis and Commentary

The review of these studies reveals a clear trend towards leveraging machine learning models in combination with environmental data to improve predictions of crop and pasture growth. Despite the promising results, several challenges persist, such as model overfitting, external variability, and the need for high-quality data inputs. Addressing these limitations requires continuous refinement of models and the inclusion of additional contextual information, such as soil characteristics and temporal variations.

By examining these studies, the present research builds on the strengths of these methodologies and aims to develop robust predictive models that can optimize pasture management and improve agricultural productivity.

Methodology

The approach taken in this study was structured to explore and analyze the influence of key environmental and temporal factors on **Total Seasonal Dry Matter (TSDM)** using machine learning techniques. The steps in the methodology include:

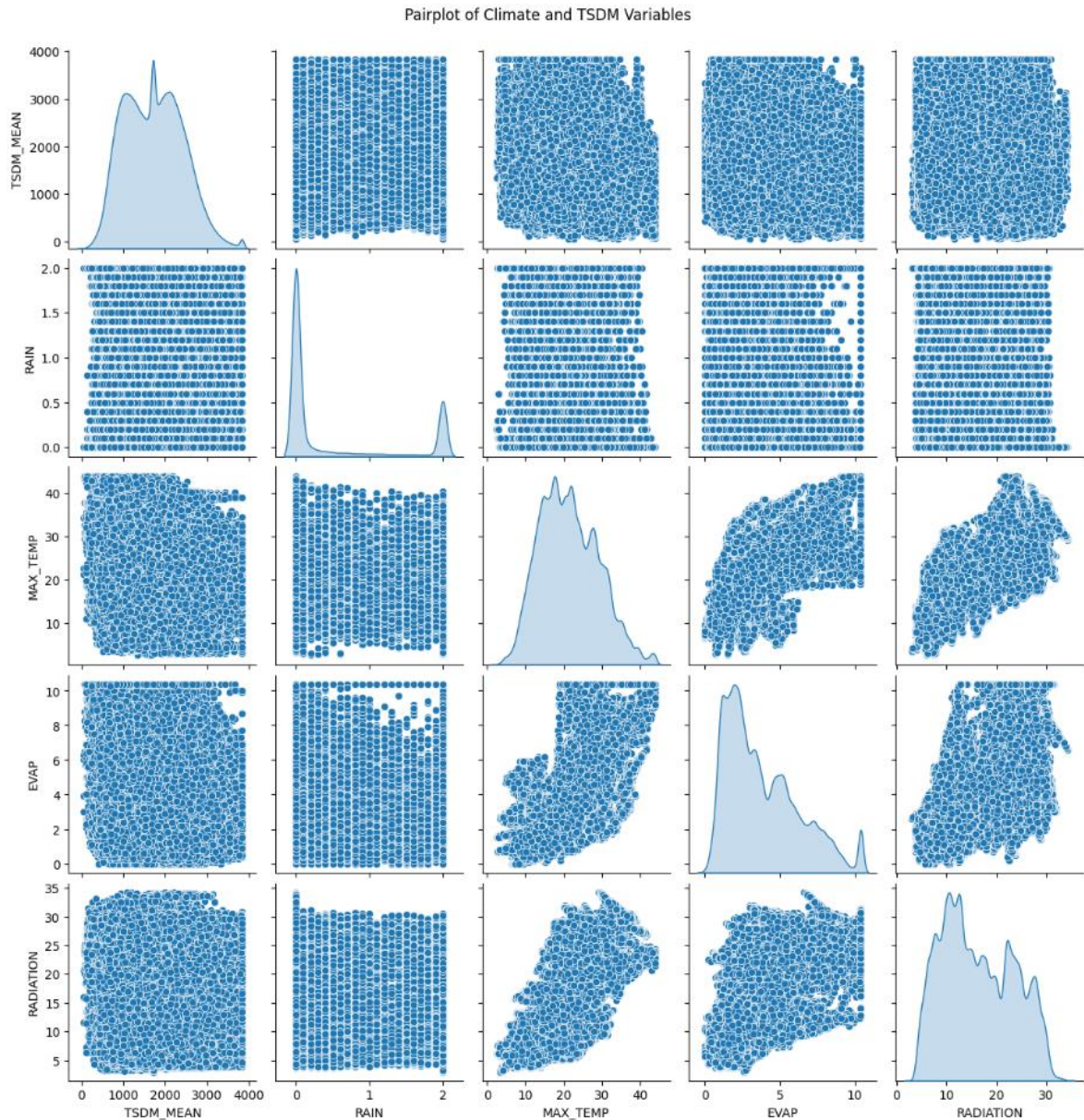
1. Data Collection:

Data was gathered from multiple datasets related to pasture growth and climate conditions. This included key environmental variables such as rainfall, temperature, relative humidity, solar radiation, and evaporation. Temporal variables such as the year and month of data collection were also incorporated. The rationale for selecting these factors is their proven impact on pasture growth, which is extensively supported by previous studies and literature.

2. Data Preprocessing & Exploratory Data Analysis (EDA):

Data preprocessing involved converting temporal information into appropriate formats, standardizing numerical variables using StandardScaler to ensure consistency, and handling missing data by imputing missing CROP_TYPE values with the mode. Outliers were identified and capped using the Interquartile Range (IQR) method to reduce their influence. Exploratory Data Analysis (EDA) was performed to identify significant patterns, detect anomalies through visualizations like box plots, and analyze correlations between variables to guide feature selection.

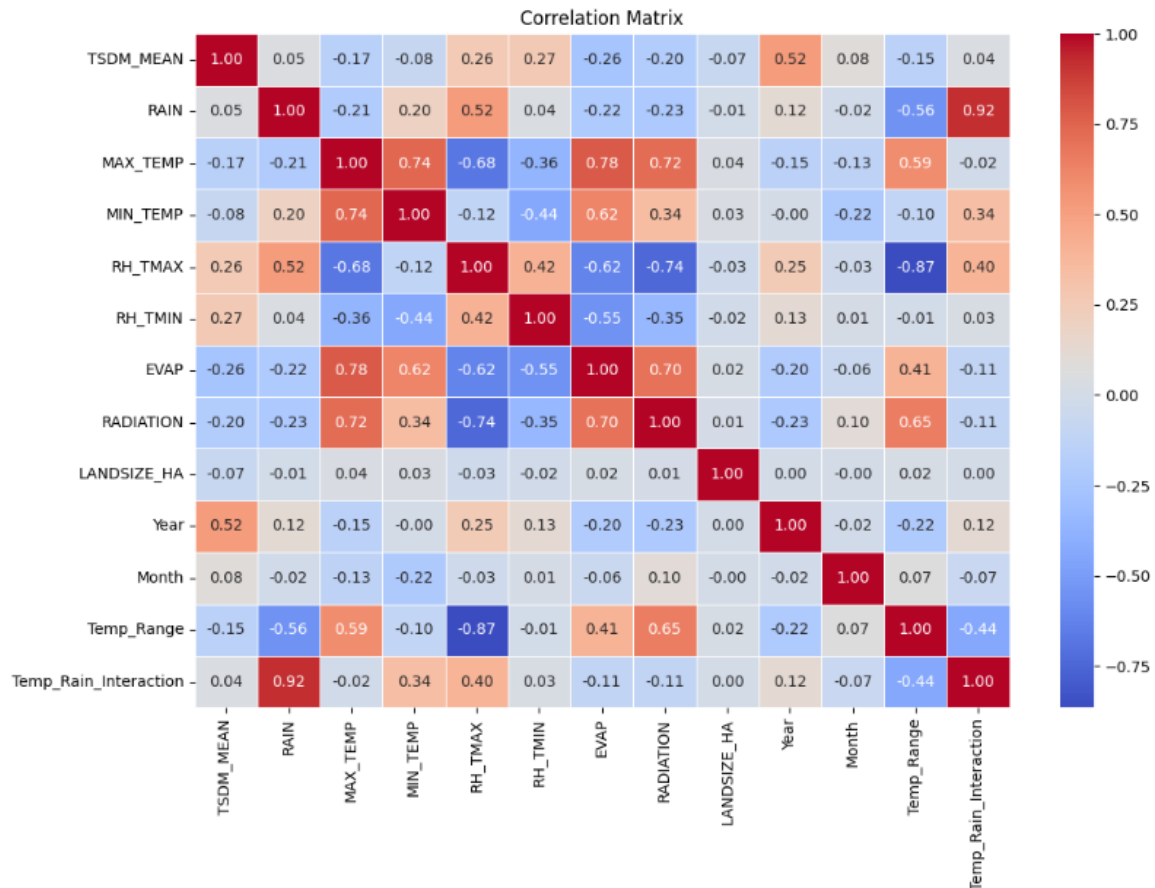
Pasture Growth Analysis Using Machine Learning



3. Feature Engineering & Selection:

Feature engineering involved creating new variables that capture interactions between critical environmental factors, such as temperature ranges and temperature-rainfall interactions. A correlation matrix was utilized to assess the relationships between variables, and only the most relevant features were selected for model training based on their predictive potential.

Pasture Growth Analysis Using Machine Learning



4. Model Selection, Training, and Hyperparameter Tuning:

Two machine learning models, Random Forest and XGBoost, were chosen for their proven effectiveness in handling complex datasets and capturing non-linear relationships. The dataset was split into an 80-20 train-test ratio to allow for robust evaluation. Hyperparameter tuning was conducted to optimize each model's performance, using techniques such as Grid Search for Random Forest and parameter selection for XGBoost.

The selection of features was carefully conducted to include those most relevant to predicting TSDM. The following features were selected based on their correlation with TSDM and their importance in representing environmental and temporal factors that influence pasture growth:

- **Year:** Represents temporal progression and seasonal effects.
- **Evaporation (EVAP):** Indicates water loss from the soil and plants, affecting soil moisture and plant growth.
- **Radiation:** Solar radiation is essential for photosynthesis and impacts overall pasture productivity.
- **Temperature Range (Temp_Range):** Captures daily temperature variations, which influence plant metabolic activities.
- **Relative Humidity at Tmax (RH_TMAX):** Reflects humidity conditions, particularly during high-temperature periods.

- **Minimum Temperature (MIN_TEMP)** and **Maximum Temperature (MAX_TEMP)**: Represent temperature extremes, which can impact growth rates.

```
# Target variable
y = numeric_data['TSDM_MEAN'] # Target variable

# Selected features
selected_feature = [
    'Year',
    'EVAP',
    'RADIATION',
    'Temp_Range',
    'RH_TMAX',
    'MIN_TEMP',
    'MAX_TEMP',
]

X_selected_df = numeric_data[selected_feature]

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_selected_df, y, test_size=0.2, random_state=42)
```

5. Model Evaluation:

Performance evaluation was conducted using several metrics including R-squared (R^2), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). Scatter plots comparing actual vs. predicted values were generated to visualize the accuracy of the predictions. Additionally, feature importance scores were extracted from the trained models to assess which variables had the greatest impact on predictions.

```
: # Evaluate the model
mse = mean_squared_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False) # RMSE is the square root of MSE
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print evaluation metrics
print(f'Mean Squared Error (MSE): {mse}')
print(f'Root Mean Squared Error (RMSE): {rmse}')
print(f'Mean Absolute Error (MAE): {mae}')
print(f'R-squared ( $R^2$ ): {r2}')
```

6. Feature importance from Random Forest Model

The Random Forest model was used to evaluate the relative importance of various features in predicting Total Seasonal Dry Matter (TSDM). Feature importance scores were generated for each variable, indicating their contribution to the model's predictive power.

7. Generating Future Predictions Using Month-Based Historical Averages:

Historical monthly averages for environmental factors were computed to generate future predictions. A range of future dates was created, and historical averages were mapped to these dates to generate input data. The trained Random Forest model was then used to predict TSDM_MEAN for the generated future dataset, allowing for insights into expected pasture growth trends.

Results

Findings

The study demonstrated the effectiveness of **Random Forest** and **XGBoost** models in predicting **Total Seasonal Dry Matter (TSDM)**, which serves as a key measure of pasture productivity. The findings include:

1. Performance of Models:

- The **Random Forest** model achieved an **R² of 0.717**, explaining approximately 72% of the variance in TSDM. This high value reflects its capability to effectively capture relationships between environmental and temporal factors influencing pasture growth.

Mean Squared Error (MSE): 0.2814475768773077
Root Mean Squared Error (RMSE): 0.5305163304529915
Mean Absolute Error (MAE): 0.38660633427178753
R-squared (R²): 0.7171190995832423

- The **XGBoost** model, on the other hand, attained an **R² of 0.664**, indicating strong predictive accuracy but slightly lower than Random Forest. This suggests that while XGBoost effectively handles complex interactions, it may not fully capture certain relationships as precisely as Random Forest.

Mean Squared Error (MSE): 0.33476189530268
Root Mean Squared Error (RMSE): 0.5785861174472474
Mean Absolute Error (MAE): 0.4348989154947407
R-squared (R²): 0.6635332681875447

2. Key Influencing Factors: Feature Importance Analysis:

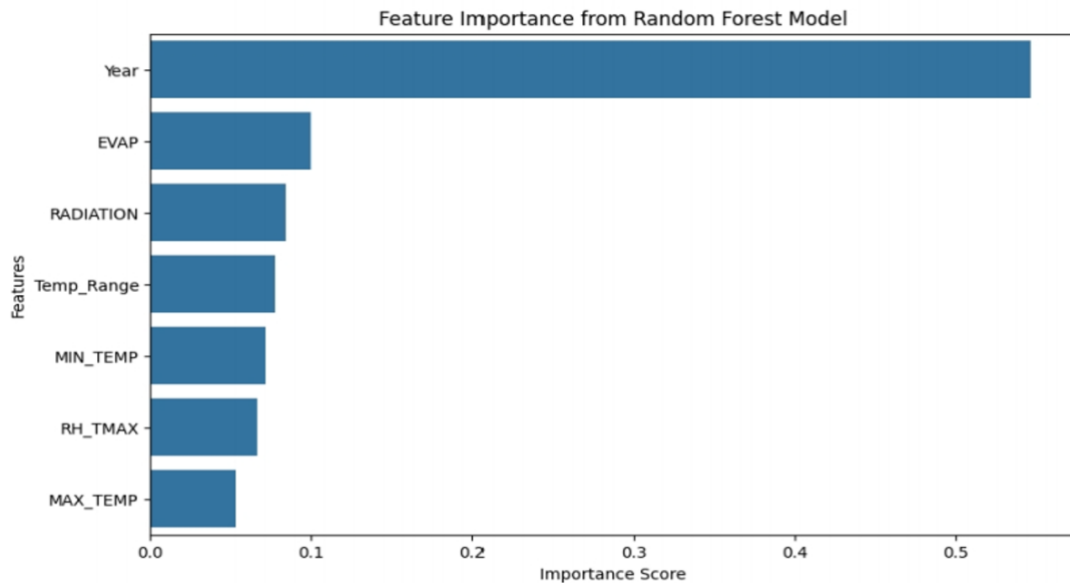
The feature importance analysis conducted on the Random Forest model identified the most influential variables in predicting Total Seasonal Dry Matter (TSDM). As illustrated in the bar plot, "Year" emerged as the most significant feature, followed by "Evaporation (EVAP)" and "Radiation." These features were critical in capturing the underlying patterns in pasture growth.

- Year:** The temporal progression represented by the "Year" variable indicates that seasonal and annual variations play a substantial role in influencing TSDM. This aligns with the idea that environmental conditions fluctuate over time, affecting pasture productivity.
- Evaporation (EVAP) and Radiation:** Both variables were shown to be crucial, reinforcing their impact on soil moisture and plant photosynthesis, respectively. Solar radiation is essential for photosynthesis, directly impacting biomass production, while evaporation reflects water loss, which affects soil moisture availability for plant growth.

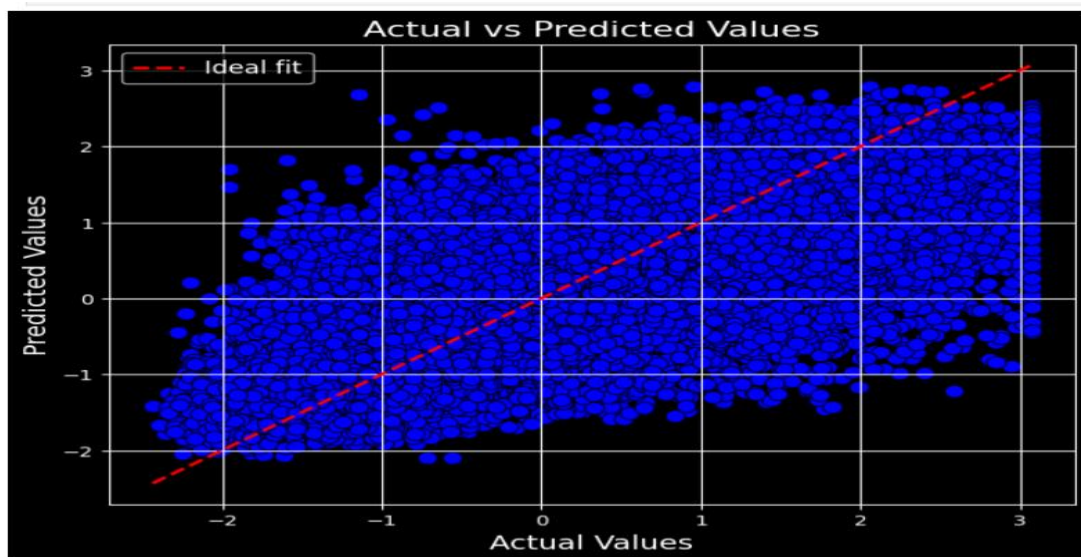
Other significant features include **Temperature Range (Temp_Range)**, **Minimum Temperature (MIN_TEMP)**, **Relative Humidity at Tmax (RH_TMAX)**, and **Maximum Temperature (MAX_TEMP)**. These variables capture additional environmental conditions that influence pasture growth rates and overall productivity.

This analysis demonstrates that specific environmental factors, including solar radiation, temperature, and moisture-related variables, are essential for accurate TSDM predictions. Understanding the contribution of these factors can aid in informed decision-making for grazing management and resource allocation, ultimately supporting sustainable agricultural practices.

Pasture Growth Analysis Using Machine Learning

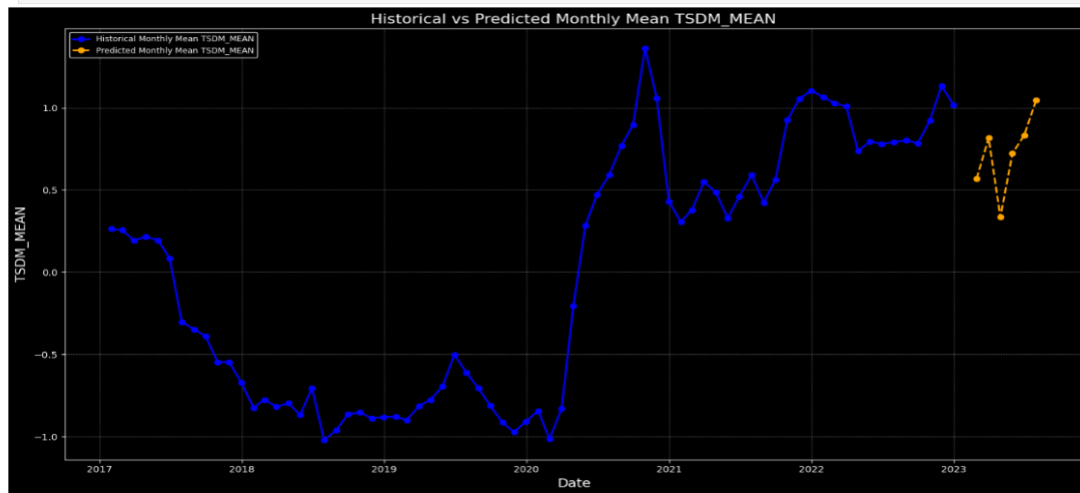


3. **Predictive Accuracy and Visual Assessment:** The models' accuracy was further confirmed through **scatter plots** comparing actual vs. predicted TSDM values. The scatter plot for the Random Forest model exhibited a strong alignment with the diagonal line, suggesting a high degree of accuracy. The XGBoost scatter plot showed a similar trend, albeit with a slightly wider spread of points, in line with its lower R^2 . These visualizations effectively validated the predictive reliability of both models.



Generating Future Data with Month-Based Historical Averages

4. **Generating Future Data:** The study also explored generating future data points based on **month-based historical averages** of key features like evaporation and temperature. This innovative approach provided insights into expected pasture growth trends over the next six months, showcasing the practical utility of these models for dynamic agricultural planning.



Significance

The findings of this study have significant implications for **livestock management** and **pasture productivity optimization**. By accurately predicting TSDM based on key environmental and temporal factors, these models provide farmers with actionable insights to make informed decisions regarding **grazing rotations**, **supplementary feeding schedules**, and **resource allocation**. The integration of these insights into farm management practices can lead to:

- **Improved livestock nutrition and health outcomes.**
- **Enhanced meat quality** through optimized feeding strategies.
- **Efficient resource utilization**, contributing to more sustainable farming practices.

Furthermore, the ability to generate reliable forecasts of future pasture growth helps farmers anticipate seasonal variations and plan accordingly, reducing uncertainties and enhancing resilience in the face of **environmental variability**.

Limitations

Despite these promising results, there are several limitations to consider:

1. **Model Generalization:** While the models performed well on the given dataset, their applicability to other regions or conditions with different environmental factors remains uncertain. Additional research is needed to validate these models across diverse agricultural contexts.
2. **Data Quality and Variability:** The predictive power of the models is highly dependent on the quality and consistency of the input data. Any inconsistencies or inaccuracies in the historical environmental data can lead to errors in predictions.
3. **Limited Feature Set:** Although key influencing factors like rainfall and temperature were included, the study did not incorporate other potential variables such as **soil health** or **moisture content**, which may also impact pasture growth. Including these additional factors could further improve the model's predictive accuracy.

Reflection

This project provided a comprehensive learning experience, revealing the challenges and intricacies of using machine learning techniques to model agricultural phenomena like pasture growth. The process taught me valuable lessons, from data preparation to model evaluation, and provided insights into potential improvements for future research.

Data Preprocessing and Feature Engineering: One of the most significant lessons learned was the importance of data preprocessing. Inconsistent data, outliers, and missing values can easily distort model performance if not carefully addressed. Techniques like standardization ensured that the model treated all variables equally, while handling outliers reduced noise in the dataset. Additionally, feature engineering, such as creating variables like temperature range and temporal averages, helped the model capture underlying patterns, enhancing predictive accuracy.

Model Selection and Optimization: Selecting suitable machine learning models and tuning them for optimal performance was a critical learning area. Random Forest and XGBoost proved effective in managing complex, non-linear datasets common in environmental data. However, I discovered that these models could easily overfit without proper tuning. Hyperparameter tuning, especially with Grid Search for Random Forest and specific parameter adjustments for XGBoost, highlighted the importance of balancing model complexity with generalizability.

Insights from Feature Importance Analysis: Analyzing feature importance provided valuable insights into the key factors influencing pasture growth, with "Year," "Evaporation," and "Radiation" emerging as significant predictors. This reinforced the idea that machine learning models can offer valuable insights into domain-specific dynamics, helping prioritize data collection and monitoring efforts. Identifying the most impactful features can guide better decision-making for resource allocation in livestock management.

Challenges in Agricultural Data Modeling: This project highlighted the unique challenges of modeling agricultural data, where variability in environmental conditions and natural randomness make it difficult to capture all influencing factors. Unlike controlled datasets, agricultural data is subject to seasonal shifts, yearly changes, and unpredictable weather events. This experience underscored the importance of adaptive models that can remain accurate over time through periodic updates with new data.

Potential for Future Research and Improvements: Based on this study, several avenues for future work emerged. Incorporating additional variables, like soil health and moisture content, could improve predictive accuracy by capturing a more comprehensive view of pasture growth factors. Exploring ensemble techniques, such as stacking multiple models, could further enhance performance, while deep learning methods may offer advantages for temporal data if larger datasets are available.

Practical Implications and Field Validation: This project also highlighted the need for real-world validation of model predictions. Field experiments that compare model outputs with observed pasture productivity under various environmental conditions would confirm the model's practical utility. This step is essential for translating predictive insights into actionable guidance for farmers, ensuring that models are reliable in real-world applications.

Reflecting on the Data-Driven Decision-Making Process: Ultimately, this project emphasized the potential of data-driven insights to support sustainable agriculture. Accurate pasture growth predictions empower farmers to make informed decisions on grazing management, feeding schedules, and resource allocation. Machine learning can improve productivity while encouraging environmentally conscious practices, making it a valuable tool for modern agriculture.

References

[1]

K. V. S. Bhai, Sk. M. Ahmad, K. Y. N. Gupta, Ch. Jyothi, and Md. Yunus, "Forecasting the Environment Factors for Optimal Plant Growth using XGBoost," in *2024 2nd International Conference on Sustainable Computing and Smart Systems (ICSCSS)*, Coimbatore, India: IEEE, Jul. 2024, pp. 71–78. doi: [10.1109/ICSCSS60660.2024.10625077](https://doi.org/10.1109/ICSCSS60660.2024.10625077).

[2]

D. De Rosa *et al.*, "Predicting pasture biomass using a statistical model and machine learning algorithm implemented with remotely sensed imagery," *Computers and Electronics in Agriculture*, vol. 180, p. 105880, Jan. 2021, doi: [10.1016/j.compag.2020.105880](https://doi.org/10.1016/j.compag.2020.105880).

[3]

Y. Guo, "Integrating genetic algorithm with ARIMA and reinforced random forest models to improve agriculture economy and yield forecasting," *Soft Comput*, vol. 28, no. 2, pp. 1685–1706, Jan. 2024, doi: [10.1007/s00500-023-09516-8](https://doi.org/10.1007/s00500-023-09516-8).

[4]

M. J. Hill, G. E. Donald, M. W. Hyder, and R. C. G. Smith, "Estimation of pasture growth rate in the south west of Western Australia from AVHRR NDVI and climate data," *Remote Sensing of Environment*, vol. 93, no. 4, pp. 528–545, Dec. 2004, doi: [10.1016/j.rse.2004.08.006](https://doi.org/10.1016/j.rse.2004.08.006).

[5]

T. Kavzoglu and A. Teke, "Predictive Performances of Ensemble Machine Learning Algorithms in Landslide Susceptibility Mapping Using Random Forest, Extreme Gradient Boosting (XGBoost) and Natural Gradient Boosting (NGBoost)," *Arab J Sci Eng*, vol. 47, no. 6, pp. 7367–7385, Jun. 2022, doi: [10.1007/s13369-022-06560-8](https://doi.org/10.1007/s13369-022-06560-8).

[6]

M.Tech, Department of Computer Science & Engineering, SRM Institute of Science & Technology, Chennai, India., R. Ravi*, Dr. B. Baranidharan, and Associate Professor, Department of Computer Science & Engineering, SRM Institute of Science & Technology, Chennai, India., "Crop Yield Prediction using XG Boost Algorithm," *IJRTE*, vol. 8, no. 5, pp. 3516–3520, Jan. 2020, doi: [10.35940/ijrte.D9547.018520](https://doi.org/10.35940/ijrte.D9547.018520).

[7]

J. N. Park, D. H. Cobon, and D. G. Phelps, “Modelling Pasture Growth in the Mitchell Grasslands”.

[8]

Y. Zhao and D. D. Justina, “Machine learning approaches for crop growth monitoring using multi-temporal and multi-variety remotely sensed data,” in *IGARSS 2020 - 2020 IEEE International Geoscience and Remote Sensing Symposium*, Waikoloa, HI, USA: IEEE, Sep. 2020, pp. 4890–4893. doi: [10.1109/IGARSS39084.2020.9324066](https://doi.org/10.1109/IGARSS39084.2020.9324066).

[9]

J. V. Silva, J. V. Heerwaarden, P. Reidsma, A. G. Laborte, K. Tesfaye, and M. K. V. Ittersum, “Big data, small explanatory and predictive power: Lessons from random forest modeling of on-farm yield variability and implications for data-driven agronomy,” *Field Crops Research*, vol. 302, p. 109063, Oct. 2023, doi: [10.1016/j.fcr.2023.109063](https://doi.org/10.1016/j.fcr.2023.109063).

Appendix 1: Supplementary Information

Step 1: Import Required Libraries

```
In [ ]: # Import necessary libraries

import pickle
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
from scipy.stats import randint
from datetime import timedelta
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.preprocessing import StandardScaler
from xgboost import XGBRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

Step 2: Load Data

```
In [ ]: # Load climate data

folder_path = r'D:\Pasture Growth\datasets\'

climate_data = pd.concat([
    pd.read_csv(f'{folder_path}climate_{year}.csv') for year in range(2017, 2023)
])

climate_data.head()

In [ ]: climate_data.shape
```

```
In [ ]: # Load TSDM data

tsdm_data = pd.read_csv(r'D:\Pasture Growth\datasets\nrm1010_tsdm.csv')

tsdm_data.head()

In [ ]: tsdm_data.shape

In [ ]: # Load paddock metadata

paddock_metadata = pd.read_csv(r'D:\Pasture Growth\datasets\paddock_metadata.csv')

paddock_metadata.head()

In [ ]: paddock_metadata.shape
```

Step 3: Data Preprocessing & EDA

```
In [ ]: # Convert DATE to datetime format

climate_data['DATE'] = pd.to_datetime(climate_data['DATE'], format='%Y-%m-%d')

tsdm_data['OBSERVATION_DATE'] = pd.to_datetime(tsdm_data['OBSERVATION_DATE'], dayfirst=True)

In [ ]: # Check unique values in PASTURE_STATE column

print(paddock_metadata['PASTURE_STATE'].unique())

In [ ]: # Filter the DataFrame where PASTURE_STATE is 'Grazing'

paddock_metadata = paddock_metadata[paddock_metadata['PASTURE_STATE'] == 'Grazing']

In [ ]: print(paddock_metadata['PASTURE_STATE'].unique())

In [ ]: paddock_metadata.shape
```

Pasture Growth Analysis Using Machine Learning

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
In [ ]: # Convert Paddock_ID to lowercase for consistency

tsdm_data['Paddock_ID'] = tsdm_data['Paddock_ID'].str.lower()
climate_data['Paddock_ID'] = climate_data['Paddock_ID'].str.lower()
paddock_metadata['Paddock_ID'] = paddock_metadata['Paddock_ID'].str.lower()

# Strip spaces from Paddock_ID

tsdm_data['Paddock_ID'] = tsdm_data['Paddock_ID'].str.strip()
climate_data['Paddock_ID'] = climate_data['Paddock_ID'].str.strip()
paddock_metadata['Paddock_ID'] = paddock_metadata['Paddock_ID'].str.strip()

# Common Paddock_ID values across all three datasets

common_ids_tsdm_climate = set(tsdm_data['Paddock_ID']).intersection(set(climate_data['Paddock_ID']))
common_ids_all = common_ids_tsdm_climate.intersection(set(paddock_metadata['Paddock_ID']))

# Number of common Paddock_IDs

print(f"Number of common Paddock_IDs between TSDM and Climate data: {len(common_ids_tsdm_climate)}")
print(f"Number of common Paddock_IDs across all datasets: {len(common_ids_all)}")

In [ ]: # Merge TSDM data with Climate data

merged_data = pd.merge(tsdm_data, climate_data,
                        left_on=['Paddock_ID', 'OBSERVATION_DATE'],
                        right_on=['Paddock_ID', 'DATE'],
                        how='inner')

# Merge with paddock metadata

merged_data = pd.merge(merged_data, paddock_metadata,
                        on='Paddock_ID',
                        how='inner')

merged_data.head()

In [ ]: merged_data.shape
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

3/17

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
In [ ]: # Remove the DATE and CREATION_DATE column from the merged_data

merged_data.drop(columns=['DATE', 'CREATION_DATE'], inplace=True)

merged_data.columns

In [ ]: merged_data.head()

In [ ]: merged_data.shape

In [ ]: merged_data.info()

In [ ]: # Count of missing values in each column

merged_data.isnull().sum()

In [ ]: # Check for outlier in TSDM_MEAN column

plt.figure(figsize=(10, 6))
sns.boxplot(x=merged_data['TSDM_MEAN'])
plt.title('Boxplot of TSDM_MEAN')
plt.show()

In [ ]: # Impute missing TSDM_MEAN values with the median

merged_data['TSDM_MEAN'].fillna(merged_data['TSDM_MEAN'].median(), inplace=True)

In [ ]: # Count of missing values in each column

merged_data.isnull().sum()

In [ ]: # Impute missing CROP_TYPE values with the mode

merged_data['CROP_TYPE'].fillna(merged_data['CROP_TYPE'].mode()[0], inplace=True)

In [ ]: # Count of missing values in each column
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

4/17

Pasture Growth Analysis Using Machine Learning

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
merged_data.isnull().sum()

In [ ]: # check for stats

merged_data.describe()

In [ ]: # Box Plot of TSDM_MEAN

plt.figure(figsize=(10, 5))
sns.boxplot(x=merged_data['TSDM_MEAN'])
plt.title('Box Plot of TSDM_MEAN')
plt.xlabel('TSDM_MEAN')
plt.show()

In [ ]: # Box Plot of RAIN

plt.figure(figsize=(10, 5))
sns.boxplot(x=merged_data['RAIN'])
plt.title('Box Plot of RAIN')
plt.xlabel('RAIN')
plt.show()

In [ ]: # Box Plot of MAX_TEMP

plt.figure(figsize=(10, 5))
sns.boxplot(x=merged_data['MAX_TEMP'])
plt.title('Box Plot of MAX_TEMP')
plt.xlabel('MAX_TEMP')
plt.show()

In [ ]: # Box Plot of MIN_TEMP

plt.figure(figsize=(10, 5))
sns.boxplot(x=merged_data['MIN_TEMP'])
plt.title('Box Plot of MIN_TEMP')
plt.xlabel('MIN_TEMP')
plt.show()
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

5/17

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
In [ ]: # Box Plot of RH_TMAX

plt.figure(figsize=(10, 5))
sns.boxplot(x=merged_data['RH_TMAX'])
plt.title('Box Plot of RH_TMAX')
plt.xlabel('RH_TMAX')
plt.show()

In [ ]: # Box Plot of RH_TMIN

plt.figure(figsize=(10, 5))
sns.boxplot(x=merged_data['RH_TMIN'])
plt.title('Box Plot of RH_TMIN')
plt.xlabel('RH_TMIN')
plt.show()

In [ ]: # Box Plot of EVAP

plt.figure(figsize=(10, 5))
sns.boxplot(x=merged_data['EVAP'])
plt.title('Box Plot of EVAP')
plt.xlabel('EVAP')
plt.show()

In [ ]: # Box Plot of RADIATION

plt.figure(figsize=(10, 5))
sns.boxplot(x=merged_data['RADIATION'])
plt.title('Box Plot of RADIATION')
plt.xlabel('RADIATION')
plt.show()

In [ ]: # Box Plot of LANDSIZE_HA

plt.figure(figsize=(10, 5))
sns.boxplot(x=merged_data['LANDSIZE_HA'])
plt.title('Box Plot of LANDSIZE_HA')
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

6/17

Pasture Growth Analysis Using Machine Learning

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
plt.xlabel('LANDSIZE_HA')
plt.show()

In [ ]: merged_data.describe()

In [ ]: merged_data.head()

In [ ]: # Function to cap outliers using IQR

def cap_outliers_iqr(merged_data):

    exclude_cols = ['MIN_TEMP', 'RH_TMAX', 'RADIATION']

    # Iterate over each numerical column in the DataFrame
    for col in merged_data.select_dtypes(include=['float64']).columns:

        if col in exclude_cols:
            continue

        Q1 = merged_data[col].quantile(0.25)
        Q3 = merged_data[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        print(f"Column: {col}")
        print(f"   Q1: {Q1}")
        print(f"   Q3: {Q3}")
        print(f"   IQR: {IQR}")
        print(f" Lower Bound: {lower_bound}")
        print(f" Upper Bound: {upper_bound}")

        # Cap the outliers
        merged_data[col] = merged_data[col].clip(lower=lower_bound, upper=upper_bound)

    return merged_data

final_merge = cap_outliers_iqr(merged_data)
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

7/17

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
In [ ]: final_merge.head()

In [ ]: final_merge.describe()

In [ ]: # Columns to plot

columns_to_plot = ['TSDM_MEAN', 'RAIN', 'MAX_TEMP', 'RH_TMIN', 'EVAP', 'LANDSIZE_HA']

# Box plots and distribution plots after capping

for column in columns_to_plot:
    plt.figure(figsize=(12, 6))

    # Box Plot
    plt.subplot(1, 2, 1)
    sns.boxplot(x=final_merge[column])
    plt.title(f'Box Plot of {column} after capping')
    plt.xlabel(column)

    # Distribution Plot
    plt.subplot(1, 2, 2)
    sns.histplot(final_merge[column], kde=True, bins=30)
    plt.title(f'Distribution Plot of {column} after capping')
    plt.xlabel(column)
    plt.ylabel('Frequency')

    plt.tight_layout()
    plt.show()

In [ ]: # Pairplot

selected_columns = ['TSDM_MEAN', 'RAIN', 'MAX_TEMP', 'EVAP', 'RADIATION']
sns.pairplot(final_merge[selected_columns], diag_kind='kde')
plt.suptitle('Pairplot of Climate and TSDM Variables', y=1.02)
plt.show()
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

8/17

Step 4: Feature Engineering

4.1 Date Features

```
In [ ]: # Convert 'OBSERVATION_DATE' to datetime
final_merge['OBSERVATION_DATE'] = pd.to_datetime(final_merge['OBSERVATION_DATE'])

# Extracting features from the date
final_merge['Year'] = final_merge['OBSERVATION_DATE'].dt.year
final_merge['Month'] = final_merge['OBSERVATION_DATE'].dt.month

In [ ]: final_merge.head()

In [ ]: # Group by year and calculate mean of TSDM_MEAN
yearly_tsdm_mean = final_merge.groupby('Year')['TSDM_MEAN'].mean()

# Line plot for TSDM Mean over time yearly
plt.figure(figsize=(10, 6))
plt.plot(yearly_tsdm_mean.index, yearly_tsdm_mean.values, marker='o', color='green', linestyle='-', linewidth=2)
plt.title('TSDM Mean Over Time (Yearly)', fontsize=14)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Mean TSDM', fontsize=12)
plt.grid(True)
plt.show()
```

4.2 Ratio Features

```
In [ ]: # Creating difference features
final_merge['Temp_Range'] = final_merge['MAX_TEMP'] - final_merge['MIN_TEMP']
```

```
In [ ]: final_merge.head()
```

4.3 Interaction Features

```
In [ ]: # Interaction feature
final_merge['Temp_Rain_Interaction'] = final_merge['MAX_TEMP'] * final_merge['RAIN']

In [ ]: final_merge.head()
```

4.4 Scaling Numerical Features

```
In [ ]: # Initialize the StandardScaler
scaler = StandardScaler()

# List of numerical columns to scale
columns_to_scale = ['TSDM_MEAN', 'RAIN', 'MAX_TEMP', 'MIN_TEMP', 'RH_TMAX',
                    'RH_TMIN', 'EVAP', 'RADIATION', 'LANDSIZE_HA',
                    'Temp_Range', 'Temp_Rain_Interaction']

# Scale the numerical columns
final_merge[columns_to_scale] = scaler.fit_transform(final_merge[columns_to_scale])
print(final_merge[columns_to_scale].head())

In [ ]: final_merge.head()

In [ ]: final_merge.info()

In [ ]: final_merge.isnull().sum()
```

Step 5: Feature Selection

Pasture Growth Analysis Using Machine Learning

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

5.1 Correlation Matrix

```
In [ ]: # Select only numeric columns for correlation analysis

numeric_data = final_merge.select_dtypes(include=['int32', 'float64'])

numeric_data.head()

In [ ]: # Calculate the correlation matrix

correlation_matrix = numeric_data.corr()

# Plotting the heatmap

plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Correlation Matrix')
plt.show()

In [ ]: target_correlations = correlation_matrix['TSDM_MEAN'].sort_values(ascending=False)

print("Feature Correlations with TSDM_MEAN:")

print(target_correlations)
```

Step 6: Modelling

```
In [ ]: # Target variable

y = numeric_data['TSDM_MEAN'] # Target variable

# Selected features

selected_feature = [
    'Year',
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

11/17

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
    'EVAP',
    'RADIATION',
    'Temp_Range',
    'RH_TMAX',
    'MIN_TEMP',
    'MAX_TEMP',
]

X_selected_df = numeric_data[selected_feature]

# Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X_selected_df, y, test_size=0.2, random_state=42)
```

6.1 Random Forest Regressor

```
In [ ]: # Initialize and train the Random Forest Regressor

rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Make predictions

y_pred = rf_model.predict(X_test)

In [ ]: # Evaluate the model

mse = mean_squared_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False) # RMSE is the square root of MSE
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print evaluation metrics
print(f'Mean Squared Error (MSE): {mse}')
print(f'Root Mean Squared Error (RMSE): {rmse}')
print(f'Mean Absolute Error (MAE): {mae}')
print(f'R-squared (R²): {r2}')

In [ ]: # Feature importances from the trained model
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

12/17

Pasture Growth Analysis Using Machine Learning

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
importances = rf_model.feature_importances_  
  
feature_importance_df = pd.DataFrame({  
    'Feature': X_train.columns,  
    'Importance': importances  
})  
  
# Sort the DataFrame by importance  
  
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)  
  
plt.figure(figsize=(10, 6))  
  
# Bar plot  
  
sns.barplot(x='Importance', y='Feature', data=feature_importance_df)  
plt.title('Feature Importance from Random Forest Model')  
plt.xlabel('Importance Score')  
plt.ylabel('Features')  
plt.show()
```

6.2 XGBoost Regressor

```
In [ ]: # Initialize and train the XGBoost Regressor  
  
xgb_model = XGBRegressor(n_estimators=100, random_state=42)  
xgb_model.fit(X_train, y_train)  
  
# Make predictions  
  
y_pred = xgb_model.predict(X_test)  
  
In [ ]: # Evaluate the model  
  
mse = mean_squared_error(y_test, y_pred)  
rmse = mean_squared_error(y_test, y_pred, squared=False) # RMSE is the square root of MSE  
mae = mean_absolute_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

13/17

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
# Print evaluation metrics  
  
print(f'Mean Squared Error (MSE): {mse}')  
print(f'Root Mean Squared Error (RMSE): {rmse}')  
print(f'Mean Absolute Error (MAE): {mae}')  
print(f'R-squared (R²): {r2}')
```

6.3 Save Random Forest Model

```
In [ ]: # Save the model  
  
with open(r'D:\Pasture Growth\datasets\random_forest_model.pkl', 'wb') as file:  
    pickle.dump(rf_model, file)  
  
In [ ]: # Load the model  
  
with open(r'D:\Pasture Growth\datasets\random_forest_model.pkl', 'rb') as file:  
    loaded_rf_model = pickle.load(file)  
  
# Loaded model to make predictions  
  
y_pred_loaded = loaded_rf_model.predict(X_test)  
  
In [ ]: # Evaluate the Loaded model  
  
mse_loaded = mean_squared_error(y_test, y_pred_loaded)  
rmse_loaded = mean_squared_error(y_test, y_pred_loaded, squared=False) # RMSE is the square root of MSE  
mae_loaded = mean_absolute_error(y_test, y_pred_loaded)  
r2_loaded = r2_score(y_test, y_pred_loaded)  
  
# Print evaluation metrics  
  
print(f'Mean Squared Error (MSE): {mse_loaded}')  
print(f'Root Mean Squared Error (RMSE): {rmse_loaded}')  
print(f'Mean Absolute Error (MAE): {mae_loaded}')  
print(f'R-squared (R²): {r2_loaded}')
```

Scatter plot of actual vs predicted values

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

14/17

Pasture Growth Analysis Using Machine Learning

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
In [ ]: plt.style.use('dark_background')

# Scatter plot of actual vs predicted values

plt.figure(figsize=(8, 6))
sns.scatterplot(x=y_test, y=y_pred_loaded, color='blue', s=80, alpha=1, edgecolor='black')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', lw=2, linestyle='--', label='Ideal fit')
plt.xlabel('Actual Values', fontsize=14)
plt.ylabel('Predicted Values', fontsize=14)
plt.title('Actual vs Predicted Values', fontsize=16)
plt.legend(fontsize=12)
plt.grid(True)
plt.show()
```

Generating Future Data with Month-Based Historical Averages

```
In [ ]: # Extract the last OBSERVATION_DATE from final_merge

last_date = pd.to_datetime(final_merge['OBSERVATION_DATE']).max()

# Generate the next 6 months' dates from the last OBSERVATION_DATE

future_dates = pd.date_range(last_date + timedelta(days=30), periods=6, freq='MS')

# Create future_data with OBSERVATION_DATE, Year, and Month columns

future_data = pd.DataFrame({
    'OBSERVATION_DATE': future_dates,
    'Year': future_dates.year,
    'Month': future_dates.month
})

# Calculate month-based historical means for each feature from numeric_data

monthly_means = numeric_data.groupby('Month').mean()

# Populate future_data with month-based means
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

15/17

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
for feature in ['EVAP', 'RADIATION', 'Temp_Range', 'RH_TMAX', 'MIN_TEMP', 'MAX_TEMP']:
    future_data[feature] = future_data['Month'].map(monthly_means[feature])

# Display future_data

print(future_data)
```

Predicting Future TSDM_MEAN Using Trained Model on Generated Data

```
In [ ]: # Select only the features required for prediction

X_future = future_data[selected_feature]

# Use the trained model to predict TSDM_MEAN for the next 6 months

future_predictions = loaded_rf_model.predict(X_future)

# Add the predictions to the future data DataFrame

future_data['TSDM_MEAN_Predicted'] = future_predictions

# Display the final DataFrame with future dates and predictions

print(future_data[['OBSERVATION_DATE', 'TSDM_MEAN_Predicted']])
```

```
In [ ]: print(future_data.columns)
```

```
In [ ]: future_data.info()
```

Historical vs Predicted Monthly Mean TSDM_MEAN

```
In [ ]: # Convert 'OBSERVATION_DATE' to datetime

final_merge['OBSERVATION_DATE'] = pd.to_datetime(final_merge['OBSERVATION_DATE'])
future_data['OBSERVATION_DATE'] = pd.to_datetime(future_data['OBSERVATION_DATE'])

# Set 'OBSERVATION_DATE' as the index for historical data
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

16/17

Pasture Growth Analysis Using Machine Learning

11/5/24, 10:40 PM

PG-UP-1 (4) (1)

```
final_merge.set_index('OBSERVATION_DATE', inplace=True)

# Calculate monthly mean for TSDM_MEAN from historical data
monthly_mean_tsdm = final_merge['TSDM_MEAN'].resample('M').mean()

future_data.set_index('OBSERVATION_DATE', inplace=True)

monthly_predicted_tsdm = future_data['TSDM_MEAN_Predicted'].resample('M').mean()

# Plotting

plt.figure(figsize=(15, 8), facecolor='black')

# Plotting Historical Monthly Mean TSDM_MEAN

plt.plot(monthly_mean_tsdm.index, monthly_mean_tsdm,
         color='blue', label='Historical Monthly Mean TSDM_MEAN', linewidth=2, marker='o', markersize=6)

# Plotting Predicted Monthly Mean TSDM_MEAN

plt.plot(monthly_predicted_tsdm.index, monthly_predicted_tsdm,
         color='orange', linestyle='--', label='Predicted Monthly Mean TSDM_MEAN', linewidth=2, marker='o', markersize=6)

plt.title("Historical vs Predicted Monthly Mean TSDM_MEAN", color='white', fontsize=16)
plt.xlabel("Date", color='white', fontsize=14)
plt.ylabel("TSDM_MEAN", color='white', fontsize=14)
plt.legend(frameon=True, loc='upper left', fontsize='small', facecolor='black', edgecolor='white')
plt.grid(color='grey', linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()

# Display the monthly means DataFrames

print("Monthly Mean TSDM_MEAN:")
print(monthly_mean_tsdm)
print("\nMonthly Predicted TSDM_MEAN:")
print(monthly_predicted_tsdm)
```

file:///C:/Users/User/Downloads/PG-UP-1 (4) (1).html

17/17